MICROBLAZE EMBEDDED SYSTEM

Table Of Contents

Introduction 2

Procedure 2

1. Hardware Platform Development in Vivado (Part 1 – LED Counter) 2
2. Software Development in Vitis (Part 1 – LED Counter).....3
3. Hardware Extension for Switches and Push Buttons (Part 2) .3
4. Software Development for Interactive Functionality (Part 2)..4

Results.....4

Conclusion.....5

Knowledge Sharing 5

Appendix.....7

APPENDIX A (Output console for LED counter) 7

APPENDIX B (Block Diagram – part 2) 7

APPENDIX C (btn_sw_led.xdc).....8

APPENDIX D (lab2b.c) 8

APPENDIX E (Output Console – second part).....10

Introduction

The purpose of this laboratory exercise was to introduce processor-based FPGA system design using the MicroBlaze soft processor within Vivado. Unlike the previous lab, which focused on purely hardware-based Verilog design, this lab demonstrated hardware-software co-design by integrating AXI-based GPIO peripherals with a soft processor and controlling on-board LEDs, switches, and push buttons using embedded C in Vitis. The first part implemented a simple software-driven LED counter to verify system functionality, while the second part extended the design to include interactive input control. This lab highlights the importance of combining programmable logic and embedded processors to build flexible and scalable FPGA-based embedded systems.

Procedure

1. Hardware Platform Development in Vivado (Part 1 – LED Counter)

1. A working directory was created in the home folder to organize all files related to Laboratory Exercise #2.
2. The Vivado environment variables were loaded on the workstation using the appropriate setup command, and Vivado 2022.1 was launched.
3. A new project was created and the 'Create Block Design' option was selected to construct the system using IP Integrator.
4. Within the block design diagram, the 'MicroBlaze' IP was added. Block Automation was executed and the processor was configured according to the lab manual specifications, including:
    - 64 KB local memory
    - Debug & UART Enabled
    - AXI peripheral port enabled
    - Interrupt controller disabled
5. The Clocking Wizard was customized to set the primary input clock source to 'Single ended clock capable pin'.
6. Connection Automation was run to configure and connect the MicroBlaze processor, BRAM controller, AXI interconnect, clocking resources, and reset logic.
7. An AXI GPIO block was added to interact with the onboard LEDs of the Zybo Z7-10 board.
8. Connection Automation was executed again, and the 'Regenerate Layout' option was selected to organize the block diagram.
9. The GPIO block was renamed to 'leds' for clarity.
10. A Constant IP block named 'VDD' was added and connected to the 'ext_rst_in' input of the Processor System Reset block. The layout was regenerated.

11. A constraints file was created to map the system clock and LED outputs to the appropriate FPGA pins.
12. The design was validated using 'Validate Design'. An HDL wrapper was created and the bitstream was generated.
13. After successful bitstream generation, the hardware design was exported as an .xsa file including the bitstream.

2. Software Development in Vitis (Part 1 – LED Counter)

14. Vitis IDE was launched and a new 'Application Project' was created.
15. The exported hardware platform (.xsa file) was selected under the 'Create a new hardware platform' option.
16. An application project name was provided, and the 'Empty Application (C)' template was selected.
17. A C program implementing a simple LED counter was written and imported into the Vitis project.
18. The hardware platform project and software system project were built successfully, generating the application binary (.elf) file.
19. The FPGA board was programmed using the 'Program Device' option.
20. The Vitis console was attached to the running hardware target to observe the output of the printf statements.

The LED counter successfully incremented from 0 to 15 continuously, and the console output verified correct operation. The results of Part 1 are shown in **APPENDIX A** **(Output console for LED counter)**. The functionality was demonstrated to the TA before proceeding to Part 2.

3. Hardware Extension for Switches and Push Buttons (Part 2)

21. To perform the second component of the lab, an additional 8-bit AXI GPIO block was added to the hardware diagram using the 'Add IP' option. The block was configured with all lines as inputs.
22. The lower four bits of this GPIO were assigned to the onboard DIP switches, and the upper four bits were assigned to the push buttons.
23. The constraints file was updated accordingly to map the new input ports to the appropriate FPGA pins. The updated constraint file is provided in **APPENDIX C** **(btn_sw_led.xdc)**.
24. The design was validated again, and a new HDL wrapper was created. The bitstream was regenerated to reflect the updated hardware configuration.
25. The updated hardware design block diagram for Part 2 is shown in **APPENDIX B** **(Block Diagram – part 2)**.

26. The hardware platform was exported again to generate an updated .xsa file.

4. Software Development for Interactive Functionality (Part 2)

27. The updated hardware platform was loaded into Vitis IDE and the hardware platform project was rebuilt to ensure synchronization between hardware and software definitions.
28. A new C program was written to maintain a COUNT variable and implement the required functionality:

- Increment COUNT when Push Button 0 is held
- Decrement COUNT when Push Button 1 is held
- Display DIP switch values when Push Button 2 is pressed
- Display COUNT value on LEDs when Push Button 3 is pressed

29. The source code for this implementation is provided in **APPENDIX D** **(lab2b.c)**.
30. The C file was imported into Vitis IDE, and the software system project was built to generate the updated binary file.
31. The FPGA board was programmed with the new binary file and the Vitis console was attached to observe the printf outputs corresponding to user interactions.

The interactive behavior was verified successfully. The output console for Part 2 is shown in **APPENDIX E** **(Output Console – second part)**. The completed system was demonstrated to the TA.

Results

The MicroBlaze-based system functioned correctly for both components of the lab on the **Zybo Z7-10 board**. In Part 1, the software-driven LED counter operated as expected. The COUNT variable incremented continuously, and its lower four bits were written to the LED GPIO peripheral. The LEDs on the Zybo Z7-10 board displayed values from 0 to 15 repeatedly, and the console output confirmed the correct hexadecimal values during execution. This verified proper GPIO initialization and memory-mapped communication between the processor and hardware. The output for Part 1 is shown in **APPENDIX A**, and the functionality was demonstrated to the TA before proceeding to Part 2.

In Part 2, an additional 8-bit GPIO was configured as input to read DIP switches and push buttons on the Zybo Z7-10 board. The updated block diagram and constraints are provided in **APPENDIX B** and **APPENDIX C**, respectively, and the modified source code is shown in **APPENDIX D**. The interactive behavior worked as required: Push Button 0 incremented COUNT, Push Button 1 decremented COUNT, Push Button 2 displayed switch values, and Push Button 3 displayed COUNT on the LEDs. The console output verifying this behavior is shown in **APPENDIX E**.

A key challenge involved correctly decoding the push button inputs from the upper nibble of the GPIO input. Initially, a mask value of 0x30 was used instead of 0x40 for Button 2, which caused incorrect behavior. By monitoring the hexadecimal values printed in the console, the issue was identified and corrected. The Vitis console was particularly helpful for debugging, as it allowed real-time verification of input values and program flow. Additionally, after modifying the hardware, regenerating the bitstream and re-exporting the hardware platform was necessary to update xparameters.h.

Overall, the final system met all functional requirements and was successfully demonstrated to the TA.

Conclusion

This laboratory exercise successfully demonstrated processor-based FPGA system design using the MicroBlaze soft processor on the Zybo Z7-10 board. By integrating AXI GPIO peripherals and developing embedded C software in Vitis, the lab reinforced the principles of hardware-software co-design and memory-mapped I/O. The transition from purely hardware-based implementation to a processor-driven system highlighted the flexibility and scalability of modern FPGA platforms. The knowledge gained from this lab is essential for understanding embedded FPGA systems, where programmable logic and software control work together to implement efficient and adaptable designs.

Knowledge Sharing

a) **In the first part, we created a delay function by implementing a counter. The goal was to update the LEDs approximately every second as we did in the previous lab. Compare the count value in this lab to the count value you used as a delay in the previous lab. If they are different, explain why. Can you determine approximately how many clock cycles are required to execute one iteration of the delay for-loop? If so, how many?**
   In Lab 1, a hardware clock divider was used with a count of approximately 125 million to divide the 125 MHz system clock down to 1 Hz. In Lab 2, the processor runs at 100 MHz, and the delay is implemented in software using a loop. Since the delay loop requires multiple instructions per iteration (increment, compare, branch), each iteration takes several clock cycles, approximately 8–10 cycles depending on pipeline behavior. Therefore, a smaller loop count such as 10,000,000 was used in software to approximate a one-second delay. The difference in count values arises because the hardware counter increments once per clock cycle, while the software loop requires multiple clock cycles per iteration.

b) **Why is the count variable in our software delay declared as volatile?**

The volatile keyword informs the compiler that the value of a variable may change unexpectedly and should not be optimized away. In the delay function, the variable delay_count is used only to create a timing delay and does not directly affect program output. Without declaring it as volatile, the compiler might detect that the loop has no observable effect and remove or optimize it during compilation. By declaring delay_count as volatile, we ensure that each increment operation is executed exactly as written, preserving the intended delay behavior in the software implementation.

c) **What does the while(1) expression in our code do?**

The while(1) statement creates an infinite loop, meaning the program runs continuously as long as the system is powered. In embedded systems, programs typically do not terminate; instead, they repeatedly monitor inputs, update outputs, and execute control logic. In this lab, the infinite loop allows the MicroBlaze processor to continuously check the status of the push buttons and switches, update the COUNT value, and write data to the LED GPIO. This behavior is conceptually similar to an always block in Verilog, where hardware logic continuously evaluates signals on every clock cycle.

d) **Compare and contrast this lab with the previous lab. Which implementation do you feel is easier? What are the advantages and disadvantages associated with a purely software implementation such as this, when compared to a purely hardware implementation such as the previous lab?**

Lab 1 focused on a purely hardware implementation using Verilog, while Lab 2 implemented the functionality in software running on the MicroBlaze processor.

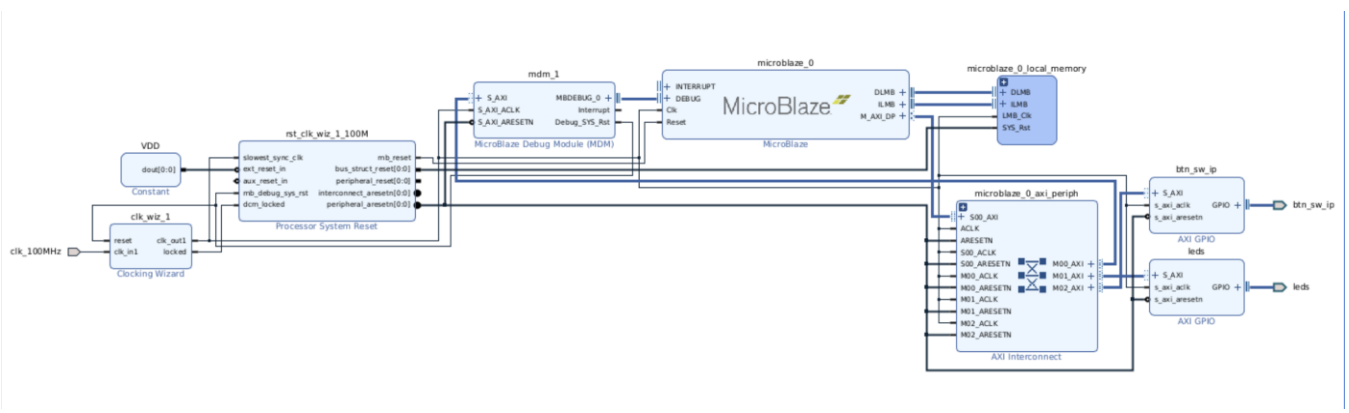
The software-based approach was easier to debug and modify, since console output and print statements provided clear visibility into program behavior. Changes required only modifying C code rather than regenerating hardware logic. However, the hardware implementation required less processor overhead and provided more precise and deterministic timing. Software implementations offer flexibility and easier development, while hardware implementations are more efficient and better suited for timing-critical applications.

Appendix

APPENDIX A (Output console for LED counter)

```
Console   Problems   Vitis Log   Guidance
TCF Debug Virtual Terminal - MicroBlaze Debug Module at USER2
Value of LEDs = 0xD
Value of LEDs = 0xE
Value of LEDs = 0xF
Value of LEDs = 0x0
Value of LEDs = 0x1
Value of LEDs = 0x2
Value of LEDs = 0x3
Value of LEDs = 0x4
Value of LEDs = 0x5
Value of LEDs = 0x6
Value of LEDs = 0x7
Value of LEDs = 0x8
Value of LEDs = 0x9
Value of LEDs = 0xA
Value of LEDs = 0xB
Value of LEDs = 0xC
Value of LEDs = 0xD
Value of LEDs = 0xE
Value of LEDs = 0xF
Value of LEDs = 0x0
Value of LEDs = 0x1
Value of LEDs = 0x2
Value of LEDs = 0x3
Value of LEDs = 0x4
Value of LEDs = 0x5
Value of LEDs = 0x6
Value of LEDs = 0x7
Value of LEDs = 0x8
Value of LEDs = 0x9
Value of LEDs = 0xA
Value of LEDs = 0xB
Value of LEDs = 0xC
Value of LEDs = 0xD
Value of LEDs = 0xE
Value of LEDs = 0xF
Value of LEDs = 0x0
Value of LEDs = 0x1
```

APPENDIX B (Block Diagram – part 2)



APPENDIX C (btn_sw_led.xdc)

Z: > Spring_2026 > ECEN749 > lab_2b_new > lab_2b_new.srcs > constrs_1 > new > btn_sw_led.xdc

```xdc
#clock_rt1
set_property PACKAGE_PIN K17 [get_ports clk_100MHz]
set_property IOSTANDARD LVCMOS33 [get_ports clk_100MHz]
create_clock -add -name sys_clk_pin -period 10.00 -waveform {0 5} [get_ports clk_100MHz]

#leds_tri_o
set_property PACKAGE_PIN M14 [get_ports {leds_tri_o[0]}]
set_property IOSTANDARD LVCMOS33 [get_ports {leds_tri_o[0]}]

set_property PACKAGE_PIN M15 [get_ports {leds_tri_o[1]}]
set_property IOSTANDARD LVCMOS33 [get_ports {leds_tri_o[1]}]

set_property PACKAGE_PIN G14 [get_ports {leds_tri_o[2]}]
set_property IOSTANDARD LVCMOS33 [get_ports {leds_tri_o[2]}]

set_property PACKAGE_PIN D18 [get_ports {leds_tri_o[3]}]
set_property IOSTANDARD LVCMOS33 [get_ports {leds_tri_o[3]}]

#btn_sw_ip_tri_i
##Switches
set_property PACKAGE_PIN G15 [get_ports {btn_sw_ip_tri_i[0]}]
set_property IOSTANDARD LVCMOS33 [get_ports {btn_sw_ip_tri_i[0]}]

set_property PACKAGE_PIN P15 [get_ports {btn_sw_ip_tri_i[1]}]
set_property IOSTANDARD LVCMOS33 [get_ports {btn_sw_ip_tri_i[1]}]

set_property PACKAGE_PIN W13 [get_ports {btn_sw_ip_tri_i[2]}]
set_property IOSTANDARD LVCMOS33 [get_ports {btn_sw_ip_tri_i[2]}]

set_property PACKAGE_PIN T16 [get_ports {btn_sw_ip_tri_i[3]}]
set_property IOSTANDARD LVCMOS33 [get_ports {btn_sw_ip_tri_i[3]}]

##Push Buttons
set_property PACKAGE_PIN K18 [get_ports {btn_sw_ip_tri_i[4]}]
set_property IOSTANDARD LVCMOS33 [get_ports {btn_sw_ip_tri_i[4]}]

set_property PACKAGE_PIN P16 [get_ports {btn_sw_ip_tri_i[5]}]
set_property IOSTANDARD LVCMOS33 [get_ports {btn_sw_ip_tri_i[5]}]

set_property PACKAGE_PIN K19 [get_ports {btn_sw_ip_tri_i[6]}]
set_property IOSTANDARD LVCMOS33 [get_ports {btn_sw_ip_tri_i[6]}]

set_property PACKAGE_PIN Y16 [get_ports {btn_sw_ip_tri_i[7]}]
set_property IOSTANDARD LVCMOS33 [get_ports {btn_sw_ip_tri_i[7]}]
```

APPENDIX D (lab2b.c)

Z: > Spring_2026 > ECEN749 > lab_2b_new > lab2b.c > main()

```c
#include <xparameters.h>
#include <xgpio.h>
#include <xstatus.h>
#include <xil_printf.h>

/* Definitions */
# define GPIO_LED_DEVICE_ID XPAR_LEDS_DEVICE_ID /* GPIO device that LEDs are connected to */
# define GPIO_INPUTS_DEVICE_ID XPAR_BTN_SW_IP_DEVICE_ID /* GPIO device that inputs (buttons/switches) are connected to */
# define WAIT_VAL 10000000

int delay (void) ;

int main ()
{
    int count ;
    int count_masked ;
    XGpio leds ;
    XGpio btn_sw;
    int status_i, status_o ;
    int sw_status, ip_state;
```

```c

    // Check the status of Push Buttons and DIP switches (inputs)
    status_i = XGpio_Initialize (&btn_sw , GPIO_INPUTS_DEVICE_ID ) ;
    XGpio_SetDataDirection(&btn_sw, 1, 0xFF ) ;
    if ( status_i != XST_SUCCESS) {
        xil_printf( "Intialization failed - Push buttons and Switches (inputs)" );
    }

    // Check the status of LEDs (outputs)
    status_o = XGpio_Initialize (&leds , GPIO_LED_DEVICE_ID ) ;
    XGpio_SetDataDirection(&leds, 1, 0x00 ) ;
    if ( status_o != XST_SUCCESS) {
        xil_printf( "Intialization failed - LEDs (outputs)" );
    }

    count = 0; //Initialize the count to 0

    while(1)
    {
        ip_state = XGpio_DiscreteRead(&btn_sw, 1);

        // Push button 0 - count increments
        if(ip_state & 0x10)   //0b 0001 0000 - To ensure BTN0 is pressed
        {
            count++;
            count_masked = count & 0xF ;
            xil_printf( "Push button 0 is pressed. Counter value = 0x%x\n\r", count_masked ) ;
            delay();
        }
        // Push button 1 - count decrements
        else if(ip_state & 0x20)   //0b 0010 0000 - To ensure BTN1 is pressed
        {
            count--;
            count_masked = count & 0xF ;
            xil_printf( "Push button 1 is pressed. Counter value = 0x%x\n\r", count_masked ) ;
            delay();
        }
```

```c
        // Push button 2 - Switch status in leds
        else if(ip_state & 0x40)   //0b 0100 0000 - To ensure BTN2 is pressed
        {
            sw_status = ip_state & 0x0F;
            XGpio_DiscreteWrite(&leds, 1, sw_status ) ;
            xil_printf( "Push button 2 is pressed. Switch status = 0x%x\n\r", sw_status ) ;
            delay();
        }
        // Push button 3 - count should be displayed
        else if(ip_state & 0x80)   //0b 1000 0000 - To ensure BTN3 is pressed
        {
            count_masked = count & 0xF ;
            XGpio_DiscreteWrite(&leds, 1, count_masked ) ;
            xil_printf( "Push button 3 is pressed. Count value = 0x%x\n\r", count_masked ) ;
            delay();
        }
    }
    return(0);
}

int delay (void)
{
    volatile int delay_count = 0;
    while ( delay_count < WAIT_VAL)
        delay_count++;
    return(0);
}
```

APPENDIX E (Output Console – second part)

Console | Problems | Vitis Log | Guidance

TCF Debug Virtual Terminal - MicroBlaze Debug Module at USER2

```
Push button 3 is pressed. Count value = 0x3
Push button 0 is pressed. Counter value = 0x4
Push button 0 is pressed. Counter value = 0x5
Push button 0 is pressed. Counter value = 0x6
Push button 2 is pressed. Switch status = 0x4
Push button 2 is pressed. Switch status = 0xC
Push button 2 is pressed. Switch status = 0xD
Push button 0 is pressed. Counter value = 0x7
Push button 0 is pressed. Counter value = 0x8
Push button 3 is pressed. Count value = 0x8
Push button 1 is pressed. Counter value = 0x7
Push button 1 is pressed. Counter value = 0x6
Push button 3 is pressed. Count value = 0x6
Push button 2 is pressed. Switch status = 0xD
Push button 0 is pressed. Counter value = 0x7
Push button 0 is pressed. Counter value = 0x8
Push button 0 is pressed. Counter value = 0x9
Push button 0 is pressed. Counter value = 0xA
Push button 3 is pressed. Count value = 0xA
Push button 1 is pressed. Counter value = 0x9
Push button 3 is pressed. Count value = 0x9
Push button 1 is pressed. Counter value = 0x8
Push button 3 is pressed. Count value = 0x8
Push button 2 is pressed. Switch status = 0x4
Push button 2 is pressed. Switch status = 0x4
Push button 2 is pressed. Switch status = 0x6
Push button 2 is pressed. Switch status = 0x7
Push button 0 is pressed. Counter value = 0x9
Push button 3 is pressed. Count value = 0x9
```